~~var~~ vs let vs const

The ~~differences~~ every interviewer wants to hear

	var	let	const
Scope	function	block	block
Hoisted	yes → undefined	yes → TDZ	yes → TDZ
Redeclare	✓ yes	✗ no	✗ no
Reassign	✓ yes	✓ yes	✗ no

TDZ = Temporal Dead Zone: the name exists, but reading it before its line throws.

1 Block scope

```
if (true) {
  var a = 1;
  let b = 2;
}
a; // 1
b; // ReferenceError
```

var ignores `{ }` blocks.
Global **var** → `window.x`

2 Hoisting & TDZ

```
x; // undefined
var x = 1;

y; // ReferenceError
let y = 1;
```

Hoisted too, but locked in the **TDZ**.
`typeof` throws too.

3 const ≠ frozen

```
const o = { n: 1 };
o.n = 2; // ok
o = {}; // TypeError

const arr = [];
arr.push(1); // ok
```

const locks the binding, not the value.

TRAP What does this log?

```
for (var i = 0; i < 3; i++)
  setTimeout(() => console.log(i));
```

`var` 3 3 3 `let` 0 1 2

let = a fresh **i** per iteration.

KEY TAKEAWAY

- `const` Your default
- `let` If it must change
- `var` Avoid it

? Follow-up questions to expect

- **Can I declare a `const` without a value?** → No: `const c;` is a SyntaxError.
- **Why does `for (const i = 0; ...; i++)` throw?** → i++ reassigns the binding: TypeError.
- **Can I mutate a `const` object?** → Yes. const locks the binding, not the value.
- **Does a top-level `var` end up on `window`?** → In classic scripts, yes. let and const never do.